Online Judge

1.Problem Statement

An **Online Judge (OJ)** is a web-based platform that allows users to solve programming problems by writing and submitting code, which is then automatically compiled, executed, and evaluated against predefined test cases. The primary goal of an OJ system is to assess the correctness and efficiency of code in an automated, fair, and scalable manner.

OJ platforms are widely used for:

- Competitive programming contests
- Interview preparation and assessments
- Skill development and practice

Well-known platforms that offer OJ functionality include **LeetCode**, **CodeChef**, **Codeforces**, and **HackerRank**.

2.Core Features

- **User Registration & Authentication**
  Secure sign-up and login, with role-based access (e.g., user, admin).
- **Problem Repository**
  A structured collection of coding problems categorized by topic, tags, and difficulty level.
- **Code Submission and Evaluation**
  Users can write or upload code. Submissions are:
    - Compiled and executed inside isolated environments (e.g., Docker containers)
    - Tested against hidden and public test cases
    - Assigned a **verdict** such as *Accepted*, *Wrong Answer*, *Time Limit Exceeded*, etc.
- **Language Support**
  Users can choose from multiple programming languages (e.g., Python, C++, Java).

- **Leaderboard and Scoring(Optional)**
  Tracks users' performance in real-time or at the end of a contest. Points are awarded based on correctness and constraints.

3.Security Consideration

To ensure fairness and prevent abuse:

- Each submission is executed inside a sandboxed container
- Time and memory limits are strictly enforced
- Malicious system commands (e.g., `rm -rf`, fork bombs) are blocked

4.Challenges and Solutions

Challenge	Solution
High concurrency (thousands of simultaneous submissions)	Use asynchronous processing with message queues to manage load efficiently
Malicious code submissions	Use Docker containers with restricted resources and no network access
Server tampering or unauthorized verdict manipulation	Use JWT-based authentication, RBAC, and logging for admin actions

5. Database Design (MongoDB Collections)

1. Users

Stores user information and credentials

```
{
  userId: String,
  fullName: String,
  email: String,
  password: String (hashed),
  dob: Date,
  createdAt: Date
}
```

2. Problems

Stores coding problem metadata and content

```
{
  name: String,
  statement: String,
  code: String (unique identifier),
  difficulty: String,
  createdAt: Date
}
```

3. TestCases

Stores test cases for each problem

```
{
  problemId: ObjectId (reference),
  input: String,
  output: String
}
```

4. Solutions

Stores user submissions and verdicts

```
{
  userId: ObjectId (reference),
  problemId: ObjectId (reference),
  code: String,
  language: String,
  verdict: String,
  submittedAt: Date
}
```

(Optional) 5. Contests

If contest support is added:

```
{
  name: String,
  startTime: Date,
  endTime: Date,
  problems: [ObjectId]
}
```

6.Architecture Overview

Frontend (React.js + Tailwind CSS)

- Home Page: View problem list
- Problem Page: View problem + code editor + submit button
- Profile Page: View past submissions
- Leaderboard Page: Rankings by score/time

Backend (Node.js + Express.js)

- RESTful APIs to handle users, problems, submissions, and contests
- Authentication using JWT
- Role-based access for users and admins

Database (MongoDB Atlas)

- Stores users, problems, test cases, and submissions

Code Execution Engine

- Receives jobs from queue
- Spins up Docker container per submission
- Compiles/runs code and compares output with test cases
- Returns verdict

Message Queue

- Used to handle asynchronous code evaluation requests and avoid overloading the backend

7.API Endpoints

1. Authentication & User Routes

a.POST /register

- Registers a new user.
- **Request**: `{ fullName, email, password, dob }`
- **Response**: `201 Created` or error

b.POST /login

- Authenticates user and returns a JWT token.
- **Request**: `{ email, password }`
- **Response**: `{ token, user }`

c.GET /profile

- Fetches profile and submission history of the currently logged-in user.
- **Headers**: `Authorization: Bearer <token>`
- **Response**: `{ user, submissions: [ ... ] }`

d.GET /submissions/:userId

- Get all submissions made by a specific user (user or admin).
- **Headers**: `Authorization`
- **Response**: `[{ problemId, verdict, language, code, submittedAt }]`

2. Problem Routes

a.GET /problems

- Lists all available problems (for home page).
- Optional query: `?difficulty=easy&tag=dp`
- **Response**: `[{ _id, name, difficulty }]`

b.GET /problems/:id

- Fetches full details of a specific problem.
- **Response**: `{ name, statement, difficulty, code }`

c.POST /problems

- **Admin Only** – Add a new problem.
- **Headers**: `Authorization`
- **Request**: `{ name, statement, code, difficulty }`
- **Response**: `201 Created`

d.PUT /problems/:id

- **Admin Only** – Update a problem.
- **Headers**: `Authorization`
- **Request**: Updated problem fields
- **Response**: `200 OK`

e.DELETE /problems/:id

- **Admin Only** – Delete a problem.
- **Headers**: `Authorization`
- **Response**: `204 No Content`

3. Submission Routes

a.POST /submit

- Submits code for evaluation.
- **Request**:

```
{
  "problemId": "<id>",
  "code": "<user_code>",
  "language": "python" | "cpp" | "java"
}
```

- **Response**: `{ submissionId, verdict }`

b.GET /submissions

- Returns all submissions of the logged-in user.
- **Headers**: `Authorization`
- **Response**: `[{ problemId, verdict, submittedAt }]`

4. Contest Routes (Optional)

a.GET /contests

- Lists all contests (past and active).
- **Response**: `[{ name, startTime, endTime }]`

b.POST /contests

- **Admin Only** – Create a contest.
- **Request**:

```
{
  "name": "April Challenge",
  "startTime": "2025-04-01T18:00:00Z",
  "endTime": "2025-04-01T21:00:00Z",
  "problems": ["<problemId1>", "<problemId2>"]
}
```

- **Response**: `201 Created`

5. Evaluation (Internal)

a.POST /evaluator/submit-job

- Internal endpoint to queue submission for evaluation (used by backend, not exposed to frontend directly).

b.POST /evaluator/verdict

- Internal endpoint to update submission with verdict after evaluation completes.

6. Admin Middleware Requirements

For all protected admin routes:

- Verify JWT
- Check if `user.role === 'admin'`